



HealthAI Coach

Synthèse des livrables — Mission MSPR

Application de suivi santé enrichie par intelligence artificielle

Document de synthèse (≈10 pages)

Frontend React · Backend Express · Microservices IA FastAPI · PostgreSQL + MongoDB

Juin 2026

Sommaire

#	Livrable	Page
1	Architecture, choix des algorithmes & APIs, benchmark frontend	3
2	Ergonomie & accessibilité (RGAA / WCAG)	4
3	Métriques de performance des modèles IA	5
4	API IA & documentation OpenAPI	6
5	Moteur de recommandation (microservices + NoSQL)	7
6	Modèle de données relationnel & adaptations	8
7	Tests automatisés & couverture	9
8	Conduite du changement & accessibilité	9
9	Maquettes d'interface responsive	10

Vue d'ensemble

HealthAI Coach est une application de suivi santé (nutrition + activité) enrichie de recommandations par IA, organisée en architecture 3-tiers + microservices.

Couche	Technologies	Port
Frontend	React 19, Vite 8, TypeScript, Tailwind CSS 4, Axios, React Router 7, Recharts	5173
Backend	Node.js, Express 4, TypeScript, Prisma 5, JWT, Zod, Swagger	5000
Base relationnelle	PostgreSQL 16 (Docker)	55432
Base NoSQL	MongoDB 7 (Docker)	27017
Microservices IA	Python 3.12, FastAPI, Ollama (LLaVA + Llama 3.2)	8001–8004



1. Architecture, algorithmes & benchmark frontend

Découpage & justification

- Le **backend agit en passerelle** (auth, quotas, validation, journalisation) et délègue l'inférence aux microservices : la charge IA (lente, CPU/GPU) est isolée du chemin critique.
- Microservices IA indépendants** : déploiement & scalabilité séparés, tolérance aux pannes, cycles de vie distincts.
- Deux bases** : PostgreSQL pour les données structurées/transactionnelles, MongoDB pour les sorties IA semi-structurées.

Choix des algorithmes IA

Capacité	Modèle / algorithme	Justification
Reconnaissance d'image	LLaVA (vision, Ollama)	Open source, local, gratuit, souveraineté des données.
Recettes / diète / sport	Llama 3.2 (Ollama)	LLM open source performant en français, sortie JSON.
Besoins caloriques	Mifflin-St Jeor (déterministe)	Formule de référence, résultat exact & reproductible (aucun LLM).
Enrichissement nutritionnel	Open Food Facts + USDA	Bases ouvertes, fiables, lient le chiffre calorique.

Calcul calorique : BMR (Mifflin-St Jeor) → TDEE (× facteur d'activité 1,2–1,9) → calories cibles (–500 perte / 0 maintien / +300 prise, plancher 1200) → macros (protéines 1,6–2,0 g/kg, lipides 0,9 g/kg, glucides = reste). La partie chiffrée est **vérifiable**, le LLM ne gère que la rédactionnel.

Benchmark frontend (choix retenus)

Dimension	Retenu	Alternatives écartées	Pourquoi
Framework UI	React 19	Vue, Angular, Svelte	Écosystème le plus large, recrutement/reprise facilités, TS mature.
Build	Vite 8	CRA, Webpack, Next.js	HMR instantané, config minimale ; SSR non requis (app authentifiée).
Styling	Tailwind CSS 4	MUI, styled-components	Design system cohérent, dark mode natif, bundle léger (purge).

Compléments : Axios (intercepteurs JWT + refresh auto), React Router 7, Recharts (graphes), react-hot-toast (notifications), lucide-react (icônes).

2. Ergonomie & accessibilité

Principes d'ergonomie (heuristiques de Nielsen, mobile-first)

- **État du système visible** : spinners IA, toasts succès/erreur, barres de progression.
- **Langage métier** en français, vocabulaire simple, emojis de repas.
- **Contrôle utilisateur** : boutons Annuler, confirmations avant suppression.
- **Cohérence** : composants UI réutilisables (Button, Input, Select, Card...).
- **Prévention des erreurs** : validations min/max, required, bornage calorique.

Navigation responsive : barre basse fixe sur mobile (<1024 px), barre latérale sur desktop (≥1024 px), grilles adaptatives 1→2 colonnes.

Accessibilité — cible WCAG 2.1 AA / RGAA 4

En place : contrastes Slate/Emerald + thème sombre, reflow sans scroll horizontal, navigation clavier (éléments natifs), focus visible, HTML sémantique, `<html lang="fr">` .

Écart constaté	Critère	Action (plan AA)
Boutons icône sans libellé	1.1.1	Ajouter <code>aria-label</code> (P1).
Labels non liés aux champs (Login)	1.3.1 / 3.3.2	<code>htmlFor</code> + <code>id</code> (P1).
Toasts non annoncés	4.1.3	Région <code>aria-live</code> (P2).
Pas de lien d'évitement	2.4.1	« Aller au contenu » (P2).

Outillage d'audit recommandé : axe-core en CI, Lighthouse (objectif ≥ 90), test clavier + lecteur d'écran (NVDA/VoiceOver).

3. Métriques de performance des modèles IA

Deux familles : un modèle **vision** (LLaVA, mesurable en précision/rappel/F1) et des modèles **génératifs** (Llama 3.2, évalués sur validité structurelle & conformité diététique). Un **harnais reproductible** est fourni dans `ai_services/evaluation/`.

Classification (reconnaissance d'image)

- **Précision** = $VP/(VP+FP)$; **Rappel** = $VP/(VP+FN)$; **F1** = moyenne harmonique.
- **Macro** = moyenne non pondérée par classe ; **Micro** = exactitude top-1 globale.

Protocole : jeu étiqueté `labels.csv` (image, classe, calories) → `python evaluate_food_recognition.py --labels ... --service-url http://localhost:8001` → `rapport_metrics.json`.

Métrique	Référence (à recalculer)
Exactitude top-1 / F1 macro / F1 micro	mesurées via le harnais sur le jeu de validation réel

Estimation calorique (LLaVA + enrichissement)

Métrique	Définition	Cible
MAE	Erreur absolue moyenne (kcal)	< 80 kcal
MAPE	Erreur relative moyenne (%)	< 25 %
Taux $\leq \pm 20$ %	Estimations dans la tolérance	> 60 %

Modèles génératifs (Llama 3.2)

- **Validité structurelle** : % de réponses JSON conformes au schéma (couvert par les tests pytest, LLM mocké) — cible 100 %.
- **Conformité diététique** : écart calories du plan vs cible TDEE déterministe — cible < 10 %.

Latence (journalisée dans `ailogs`)

Modèle	Tâche	Latence locale
LLaVA	Analyse d'image	60 s – 3 min
Llama 3.2	Génération texte	30 – 90 s
Déterministe	Calcul macros	< 50 ms

4. API IA & documentation OpenAPI

L'API IA est exposée **via le backend Express** (surface unique `/api/ai/*`, protégée JWT, quota 60 req/h), qui proxy 4 microservices FastAPI. Spec **OpenAPI 3.0** générée depuis le code (`backend/src/docs/swagger.ts`), exportée dans `docs/openapi.json` (`npm run openapi:export`), UI interactive sur `/api-docs`.

Endpoints IA

Méthode	Endpoint	Service	Modèle
POST	/api/ai/analyze-food-image	8001	LLaVA
GET	/api/ai/recipes/suggest?mealType=	8002	Llama 3.2
POST	/api/ai/recipes/generate	8002	Llama 3.2
GET	/api/ai/diet/macros	8003	déterministe
GET	/api/ai/diet/plan	8003	Llama 3.2
GET	/api/ai/diet/analyze?days=	8003	Llama 3.2
GET	/api/ai/training/program	8004	Llama 3.2
POST	/api/ai/training/quick-workout	8004	Llama 3.2
GET	/api/ai/history?type=&limit=	—	MongoDB

Technologies additionnelles (justification)

Techno	Rôle
FastAPI	Async natif (appels LLM longs), validation Pydantic, Swagger auto.
Ollama	Runtime LLM local, gratuit, souverain.
Motor / httpx	Accès MongoDB & HTTP non bloquants (async).
express-rate-limit	Quotas protégeant les ressources d'inférence.

```
POST /api/ai/diet/macros → { bmr:1780, tdee:2759, calories:2759,  
                             protein_g:150, carbs_g:344, fat_g:92 }
```

5. Moteur de recommandation (microservices + NoSQL)

4 microservices Python FastAPI **séparés** de l'application principale, produisant des recommandations personnalisées et **persistées en base NoSQL (MongoDB)**. Couplage uniquement par HTTP et par la base partagée — aucun code applicatif commun avec le backend.

Service	Port	Modèle	Rôle
api1_food_recognition	8001	LLaVA	Analyse d'image + enrichissement OFF/USDA
api2_recipe_suggestions	8002	Llama 3.2	Suggestions / génération de recettes
api3_diet_plan	8003	Llama 3.2 + TDEE	Macros & plan diététique
api4_training_program	8004	Llama 3.2	Programmes & séances express

Pourquoi NoSQL

Les sorties IA sont **hétérogènes** (recette, plan, programme, analyse) : MongoDB stocke ces documents JSON sans migration de schéma.

Collections MongoDB

Collection	Contenu
recommendations	userId, type, prompt, content (JSON), aiModel, tokens, dates
ailogs	userId, requestId, service, status, input, output, error, latencyMs

Backend (Node) et microservices (Python) écrivent dans les **mêmes collections** de la base `healthai`. URI Mongo forcée en IPv4 (127.0.0.1) pour éviter la résolution `::1` vers un conteneur distinct (Windows). Sériàlisation via `json.dumps(ensure_ascii=False)` → rendu structuré de l'historique côté front.

```
Client -JWT→ Backend /api/ai/* -HTTP→ Microservice :  
  lit profil (PostgreSQL) → prompt personnalisé → Ollama (ou calcul déterministe)  
  → normalise/enrichit → persiste (MongoDB) → réponse normalisée
```

6. Modèle de données relationnel & adaptations

PostgreSQL (Prisma) pour le métier structuré ; MongoDB pour les sorties IA. Rattachement par `userId` (UUID).

Table	Rôle	Points clés
User	Profil + identité	email unique ; champs santé optionnels ; dailyCalorieTarget
FoodEntry	Repas	calories + macros ; mealType (enum) ; index (userId, date)
ActivityEntry	Activités	duration, caloriesBurned, type ; index (userId, date)
GoalSettings	Objectifs	relation 1-1 ; cibles macros & séances
HealthMetric	Mesures corporelles	poids/IMC/masse grasse historisés
RefreshToken	Sessions JWT	rotation + révocation au logout

Intégrité : toutes les relations enfant en `onDelete: Cascade` → suppression de compte = effacement complet (RGPD).

Adaptations du modèle existant

Adaptation	Où	Motivation
Collections NoSQL recommendations & alogs	MongoDB	Sorties IA au schéma variable sans migration.
dailyCalorieTarget = limite calorique	PostgreSQL	Recalculé dynamiquement (Mifflin-St Jeor).
Objectif calories brûlées (localStorage)	Frontend	Donnée d'affichage dérivée du BMR ; évite une migration.
Champs santé User optionnels	PostgreSQL	Inscription puis onboarding progressif.
aiModel / tokens / nutrition_basis	MongoDB	Traçabilité du modèle, coût et source des valeurs.

7. Tests automatisés & couverture

Périmètre	Framework	Statut
Backend (services critiques)	Jest + Supertest	✓ En place (auth, food, activity)
Microservices IA (moteur de reco)	Pytest + mocks (~136 tests)	✓ En place
Frontend (UI)	Vitest + Testing Library	⊗ Setup fourni, à implémenter

Couverture : backend `npm run test:coverage` → `coverage/lcov-report` ; microservices `pytest --cov --cov-report=html` → `htmlcov/` . Les tests IA mockent Ollama/OFF/USDA/MongoDB (déterministes, sans réseau). CI recommandée : lint → tests back → tests IA → tests front → build → publication LCOV.

Cibles UI prioritaires : logique pure `mappers.ts` (objectifs caloriques), `Login` , `FoodLog` (analyse photo mockée), `Coach` (historique), `AppContext` .

8. Conduite du changement & adoption

Personas : grand public, sportif, développeur/partenaire, data scientist, product manager, personne en situation de handicap — chacun adressé (parcours guidé, OpenAPI, harnais d'éval, métriques, base accessible).

- **Accessibilité = levier d'adoption** : backlog priorisé effort/impact (aria-label & labels P1).
- **Documentation multi-niveaux** (utilisateur, technique, démarrage reproductible) + utilisateur de démo (seed).
- **Déploiement progressif** : bêta interne → itération → ouverture ; boucle de feedback via `ailogs` .
- **Éthique & conformité** : IA locale (souveraineté), RGPD (effacement en cascade), traçabilité du modèle.
- **Indicateurs** : complétion onboarding, reco/utilisateur, latence/erreurs IA, score Lighthouse ≥ 90, rétention 7/30 j.

9. Maquettes d'interface responsive

Système de design & breakpoints

Primaire Emerald #10b981 , accent Orange #f97316 , neutres Slate, thèmes clair/sombre. Composants : Button, Input, Select, Card, ProgressBar, Slider. Icônes lucide-react.

Breakpoint	Largeur	Comportement
Mobile	< 640 px	1 colonne, navigation basse fixe
md	≥ 768 px	grilles 2 colonnes partielles
lg	≥ 1024 px	barre latérale, tableau de bord 2 colonnes

Écrans

Login
Carte centrée : e-mail, mot de passe (œil), bascule connexion/inscription.

Onboarding
3 étapes (profil, mensurations, objectif) + barre de progression.

Dashboard
Calories (consommées/limite), brûlées/objectif, minutes actives, séances, IMC, graphe semaine.

Journal alimentaire
Ajout rapide, photo repas IA, repas groupés par type.

Coach IA
Onglets Recettes / Diète / Sport / Historique, cartes structurées.

Profil
Données, objectif, calcul IA, stats, thème, déconnexion, suppression compte.

MOBILE

Bonjour ... 🍌
Calories ███
Brûlées ███
Graphe semaine
[Accueil][Repas
...][Profil]

DESKTOP ≥ lg

Sidebar

Bonjour ... 🍌
Accueil
Repas
Coach
Profil
🍌 thème

Calories (2 col.)
Actif
Séances
IMC
Graphe semaine

Les tokens de design sont dérivables du code Tailwind → cohérence maquette ↔ implémentation. Captures réelles : `npm run dev` puis export en vues 375 px (mobile) et 1440 px (desktop).